ECE 2011
Introduction to Computer Engineering
Spring 2026

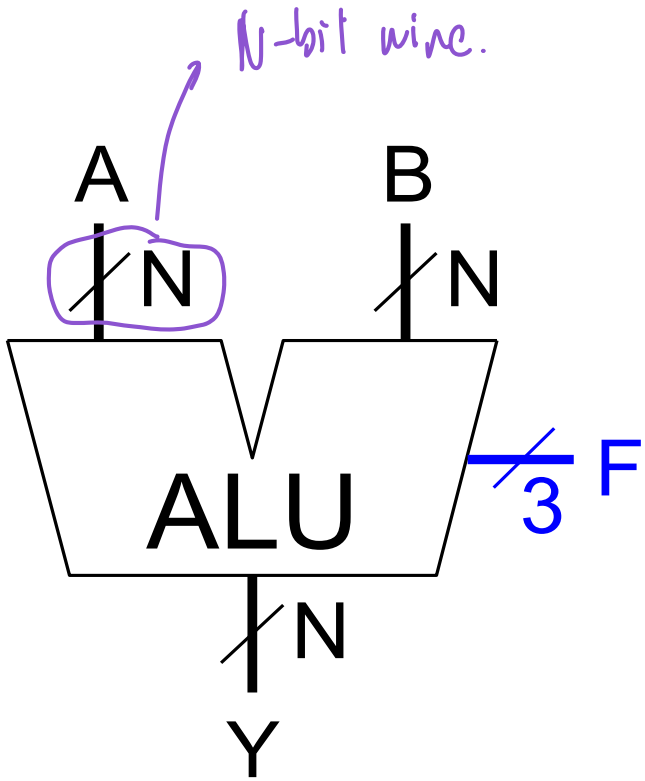
Lecture 13
02-26-2026

Linghao Song
Yale ECE

Problem Set 1

- Problem Set 1:
  - March 5 Thursday
  - Select a slot of 75 min at your convenience
  - Boolean Minimization, Finite State Machine, Timing Analysis, Understanding Verilog
  - Prepare your cheat sheet: up to 4 pages (not 4 sheets, pieces of papers), scan or take clear photos of your cheat sheet and solutions
  - Submission no later than 11:59pm ET March 5, submit to Canvas, so you should start working on your pset no later than 10:44pm.
  - No discussion, internet access, textbook, or slides.

Arithmetic Logic Unit (ALU)

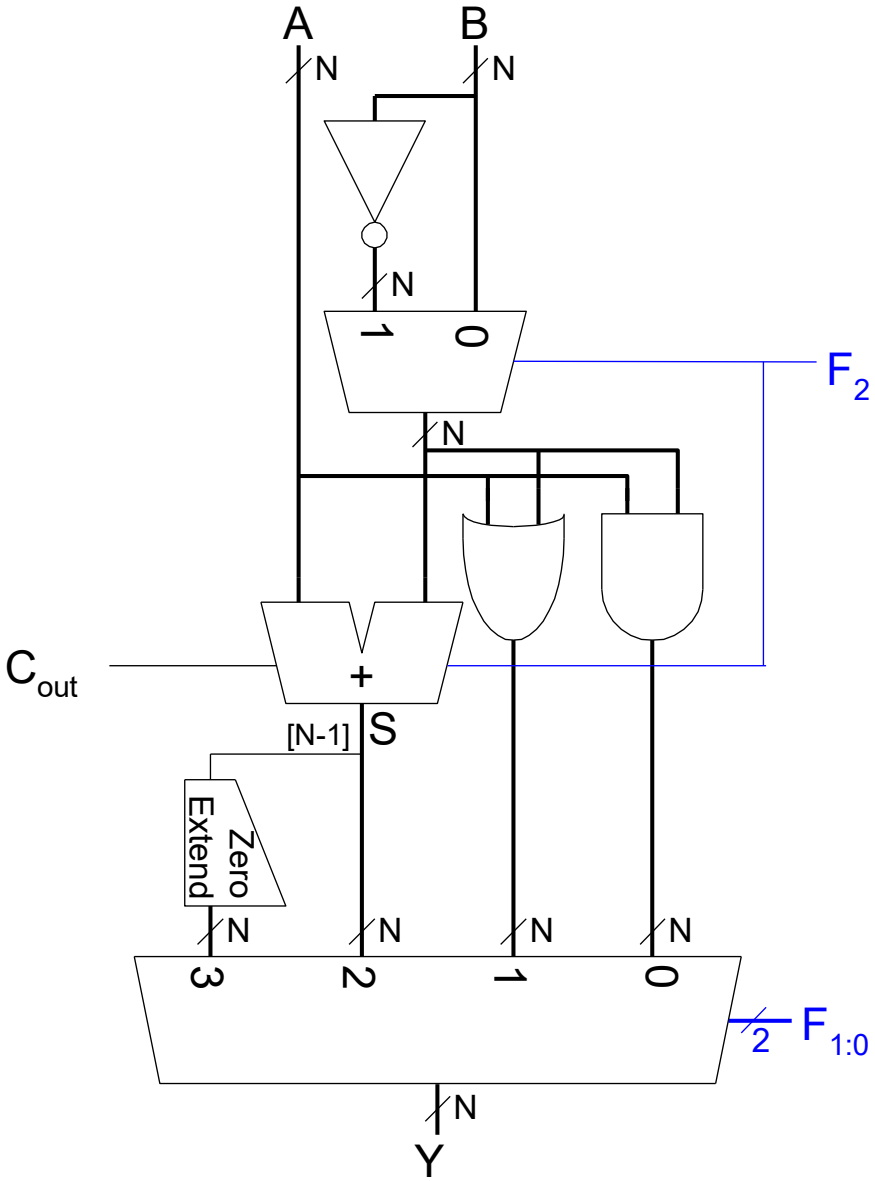


$F_{2:0}$	Function
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

ALU Design

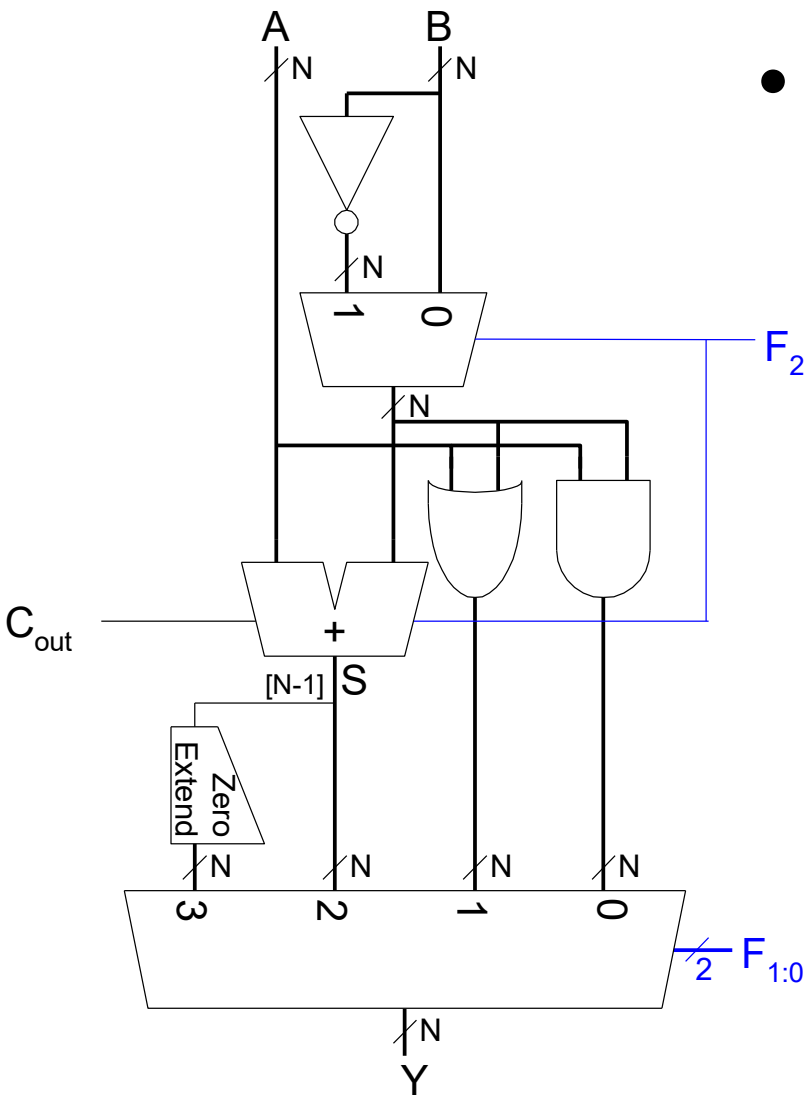


$F_{2:0}$	Function
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Set Less Than (SLT) Example



- Configure 32-bit ALU for SLT operation: *A* = 25 and *B* = 32
  - ***A* < *B***, so *Y* should be 32-bit representation of 1 (0x00000001)
  - $\boldsymbol{F_{2:0} = 111}$
    - $\boldsymbol{F_2 = 1}$ (adder acts as subtracter), so 25 - 32 = -7
    - -7 has 1 in the most significant bit ($S_{31} = 1$)
    - $\boldsymbol{F_{1:0} = 11}$ multiplexer selects *Y* = S_{31} (zero extended) = 0x00000001.

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Shifters

- **Logical shifter:** shifts value to left or right and fills empty spaces with 0's
  - Ex: **11001** >> 2 = **00110**
  - Ex: **11001** << 2 = **00100**

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Shifters

- **Logical shifter:** shifts value to left or right and fills empty spaces with 0's
  - Ex: **11001** >> 2 = **00110**
  - Ex: **11001** << 2 = **00100**

- **Arithmetic shifter:** same as logical shifter, but on right shift, fills empty spaces with the old most significant bit (msb).
  - Ex: **11001** >>> 2 = **11110** → interesting...
  - Ex: **11001** <<< 2 = **00100**

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Shifters

- **Logical shifter:** shifts value to left or right and fills empty spaces with 0's
  - Ex: **11001** >> 2 = **00110**
  - Ex: **11001** << 2 = **00100**

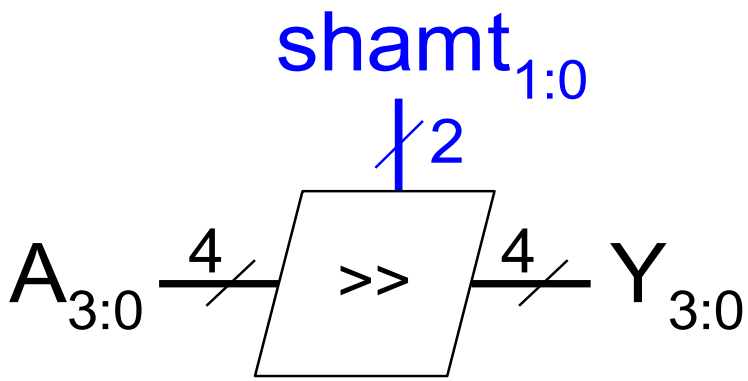
- **Arithmetic shifter:** same as logical shifter, but on right shift, fills empty spaces with the old most significant bit (msb).
  - Ex: **11001** >>> 2 = **11110**
  - Ex: **11001** <<< 2 = **00100**

- **Rotator:** rotates bits in a circle, such that bits shifted off one end are shifted into the other end
  - Ex: **11001** ROR 2 = **01110**
  - Ex: **11001** ROL 2 = **00111**

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

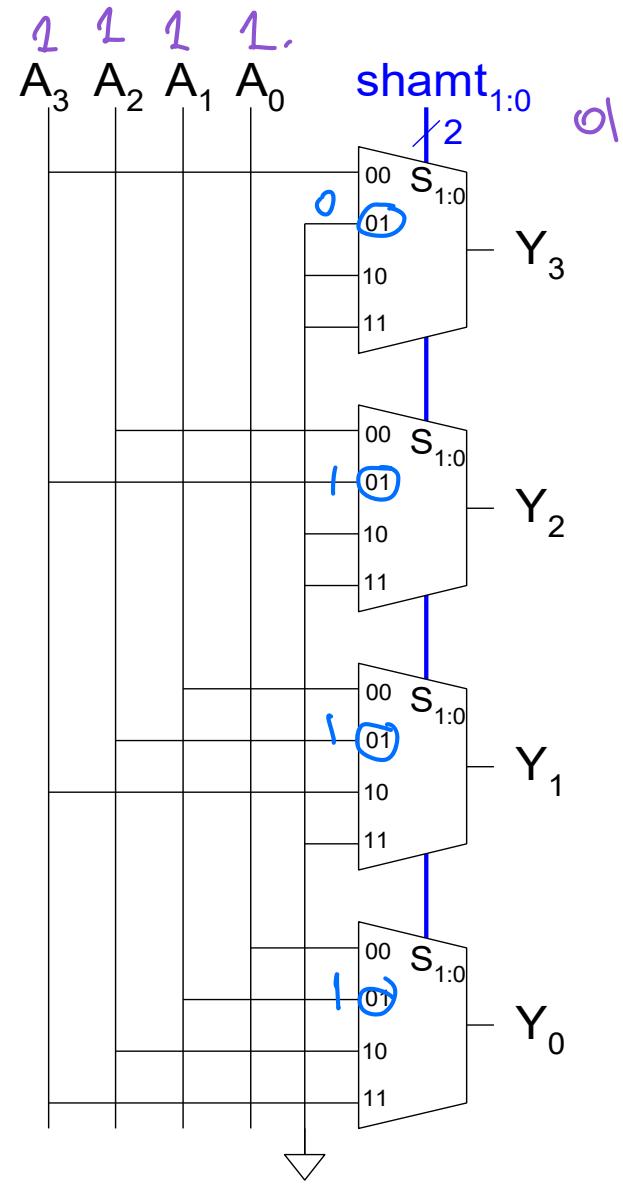
ELSEVIER

Shifter Design



this is a logic rightshift 2

1111 → 0111



© Digital Design and Computer Architecture, 2nd Edition, 2012

ELSEVIER

Shifters as Multipliers, Dividers

overflow could happen:

00001 << 4 = 10000

1 -16

- $A << N = A \times 2^N$
  - **Example:** 00001 << 2 = 00100 ($1 \times 2^2 = 4$)
  - **Example:** 11101 << 2 = 10100 ($-3 \times 2^2 = -12$)
- $A >>> N = A \div 2^N$
  - **Example:** 01000 >>> 2 = 00010 ($8 \div 2^2 = 2$)
  - **Example:** 10000 >>> 2 = 11100 ($-16 \div 2^2 = -4$)

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Multipliers

- **Partial products** formed by multiplying a single digit of the multiplier with multiplicand
- **Shifted** partial products **summed** to form result

Decimal		**Binary**
230	multiplicand	0101
x 42	multiplier	x 0111
460	partial products	0101
+ 920		0101
		0101
		+ 0000
9660	result	0100011

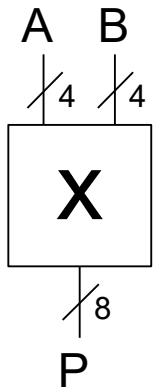
230 x 42 = 9660

5 x 7 = 35

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

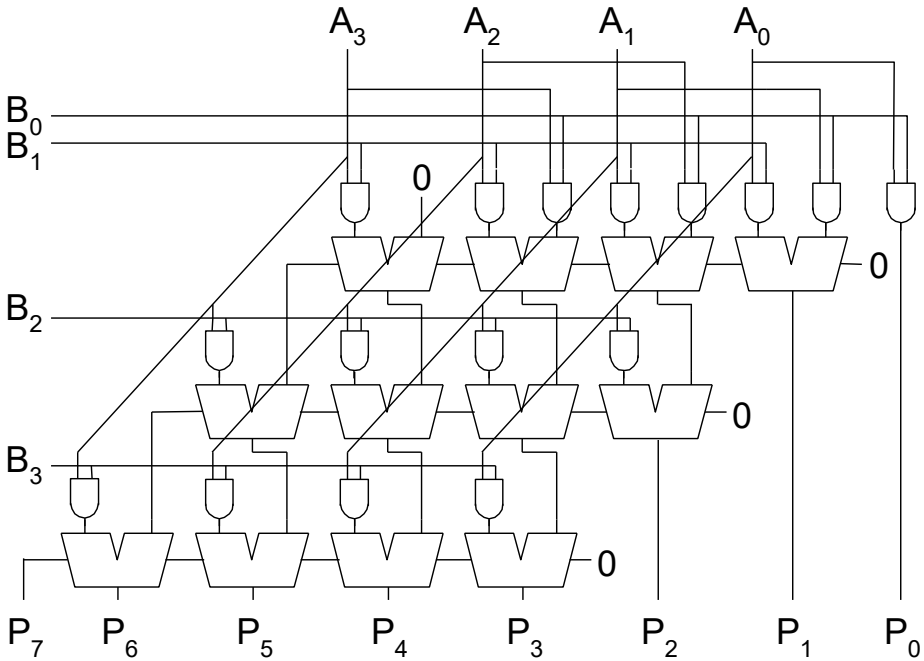
ELSEVIER

4 x 4 Multiplier



this is so cool!

$$
\begin{array}{rcccccccc}
 & & & & A_3 & A_2 & A_1 & A_0 \\
 & & & \times & B_3 & B_2 & B_1 & B_0 \\
\hline
 & & & & A_3B_0 & A_2B_0 & A_1B_0 & A_0B_0 \\
 & & & A_3B_1 & A_2B_1 & A_1B_1 & A_0B_1 & \\
 & & A_3B_2 & A_2B_2 & A_1B_2 & A_0B_2 & & \\
+ & A_3B_3 & A_2B_3 & A_1B_3 & A_0B_3 & & & \\
\hline
P_7 & P_6 & P_5 & P_4 & P_3 & P_2 & P_1 & P_0
\end{array}
$$



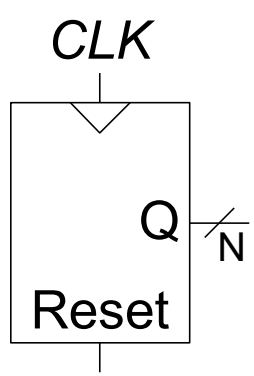
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

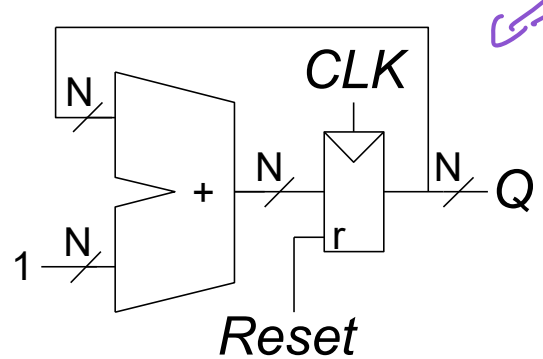
ELSEVIER

Counters

- Increments on each clock edge
- Used to cycle through numbers. For example,
  - 000, 001, 010, 011, 100, 101, 110, 111, 000, 001…
- Example uses:
  - Digital clock displays
  - Program counter: keeps track of current instruction executing

Symbol **Implementation**





this is a counter!

© Digital Design and Computer Architecture, 2nd Edition, 2012

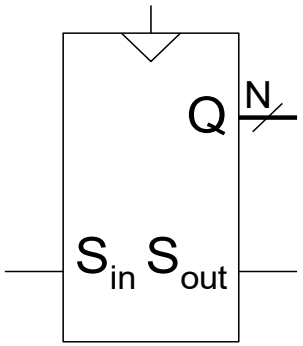
ELSEVIER

Shift Registers

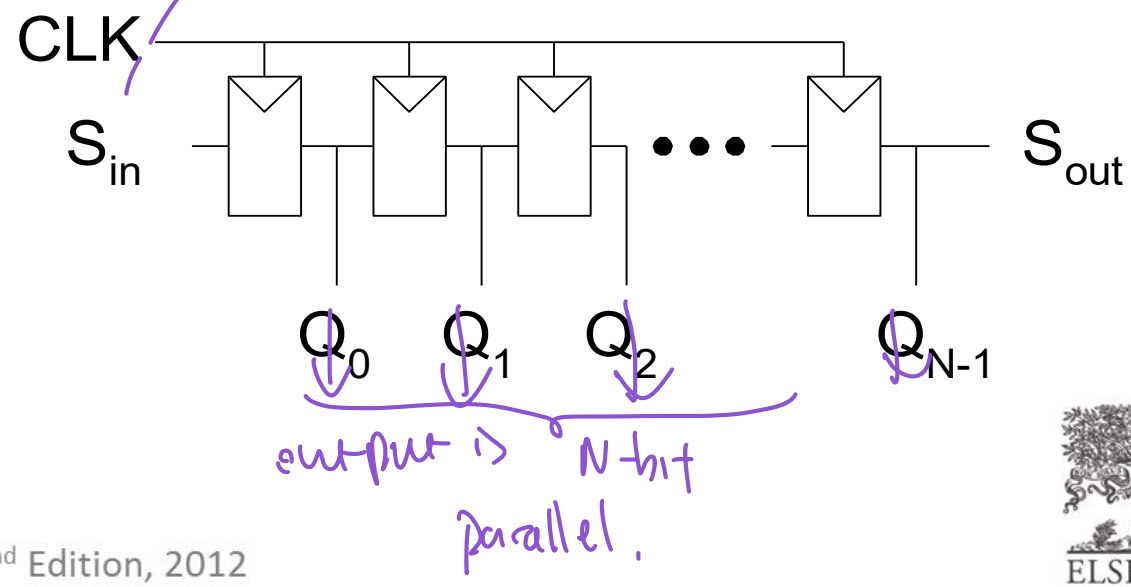
- Shift a new bit in on each clock edge
- Shift a bit out on each clock edge
- *Serial-to-parallel converter*: converts serial input (S_{in}) to parallel output ($Q_{0:N-1}$)

Input is N bit serial.

Symbol:



Implementation:



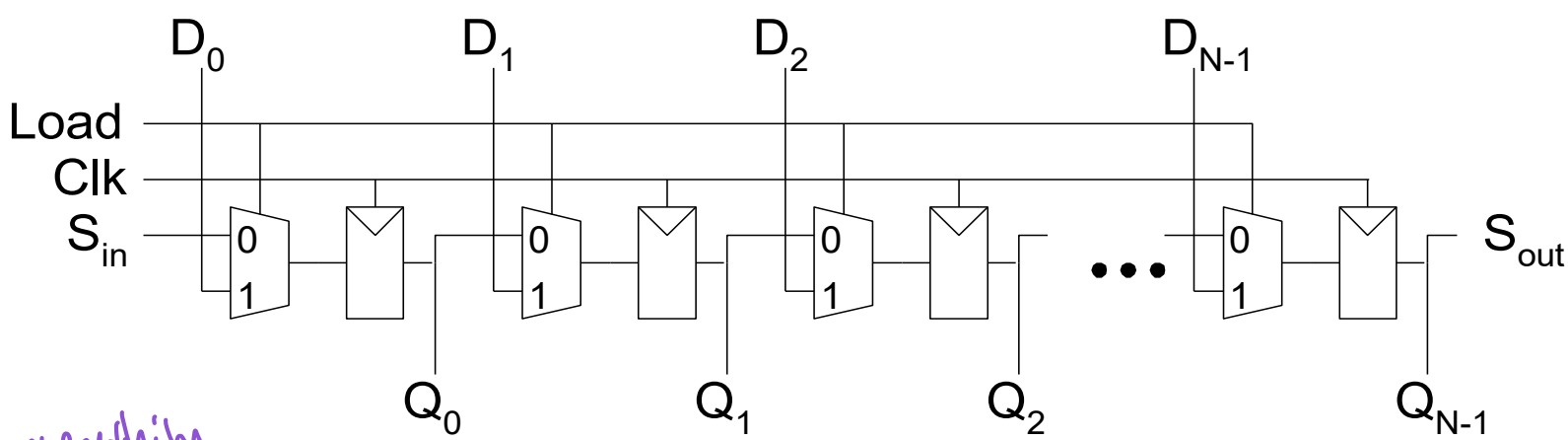
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Shift Register with Parallel Load

- When *Load* = 1, acts as a normal *N*-bit register
- When *Load* = 0, acts as a shift register
- Now can act as a *serial-to-parallel converter* (S_{in} to $Q_{0:N-1}$) or a *parallel-to-serial converter* ($D_{0:N-1}$ to S_{out})

this is very cool!



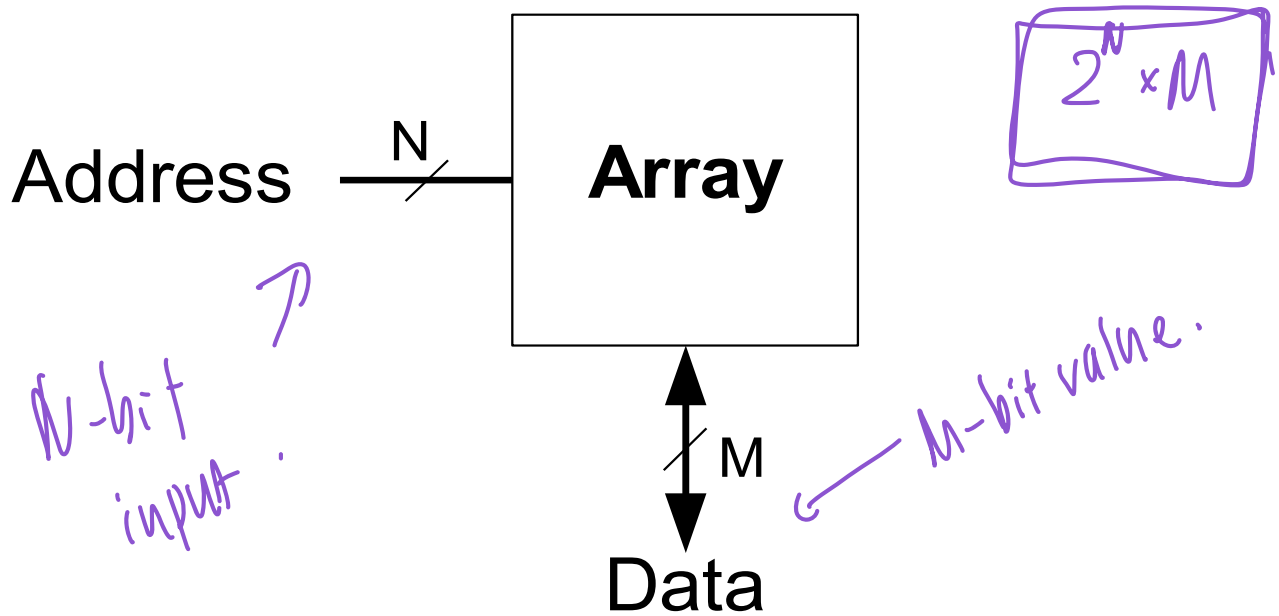
Now can do everything that can be built from +, −, ×, ÷! → shifter + ALU.

© Digital Design and Computer Architecture, 2nd Edition, 2012

ELSEVIER

Memory Arrays

- Efficiently store large amounts of data
- 3 common types:
  - Dynamic random access memory (DRAM)
  - Static random access memory (SRAM) → BRAM, URAM.
  - Read only memory (ROM)
- *M*-bit data value read/ written at each unique *N*-bit address

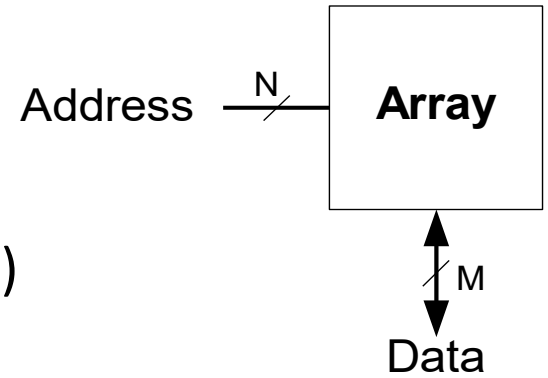


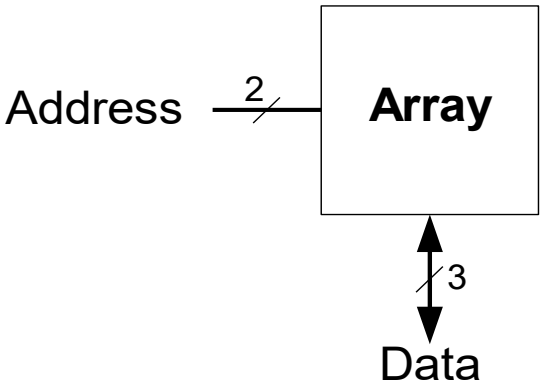
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Memory Arrays

- 2-dimensional array of bit cells
- Each bit cell stores one bit
- *N* address bits and *M* data bits:
  - 2^N rows and *M* columns
  - **Depth:** number of rows (number of words)
  - **Width:** number of columns (size of word)
  - **Array size:** depth × width = $2^N \times M$





Address	Data		
11	0	1	0
10	1	0	0
01	1	1	0
00	0	1	1

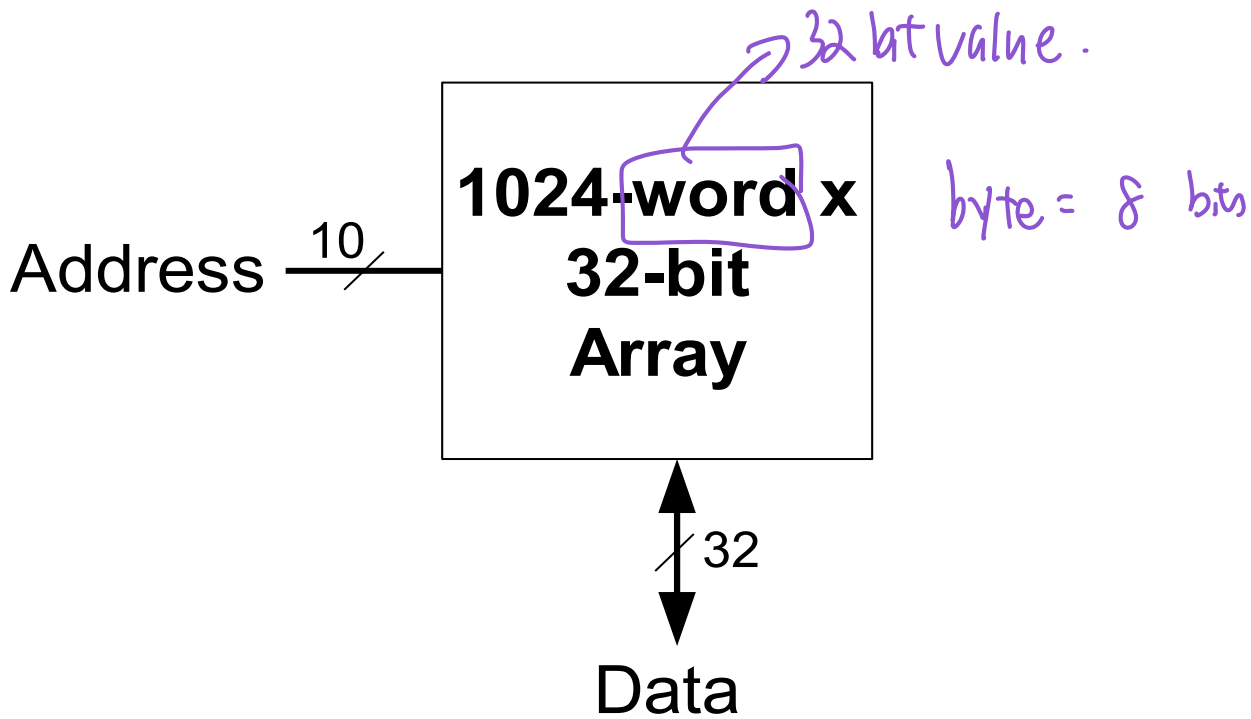
depth

width

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Memory Arrays

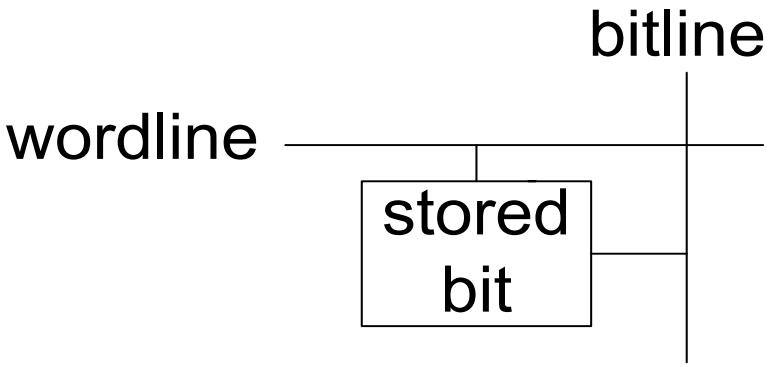


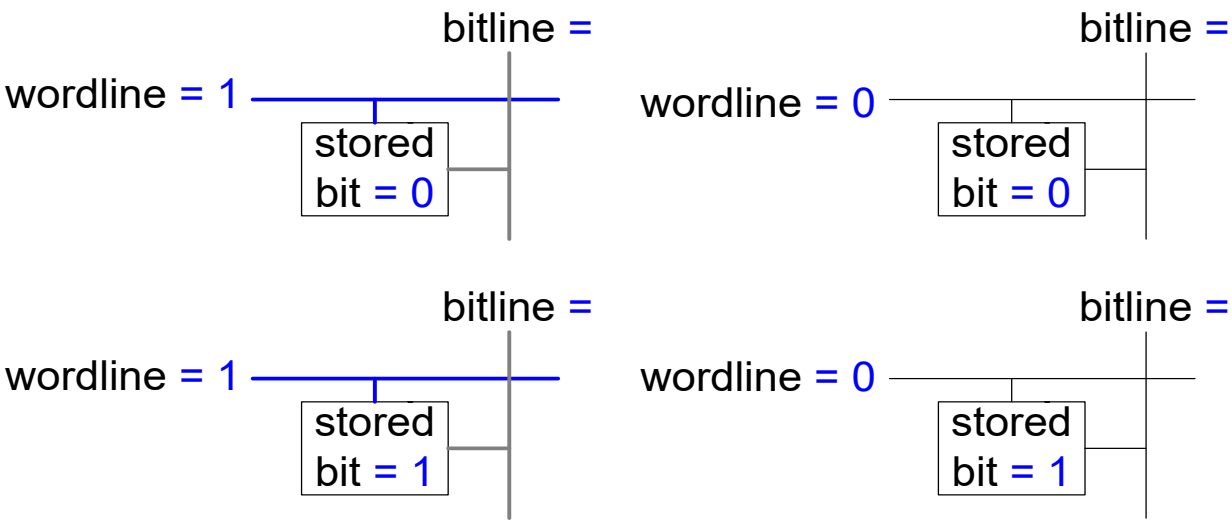
$2^{10} \times 32 = 2^{10} \times 2^{5} = 2^{15}$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Memory Array Bit Cells



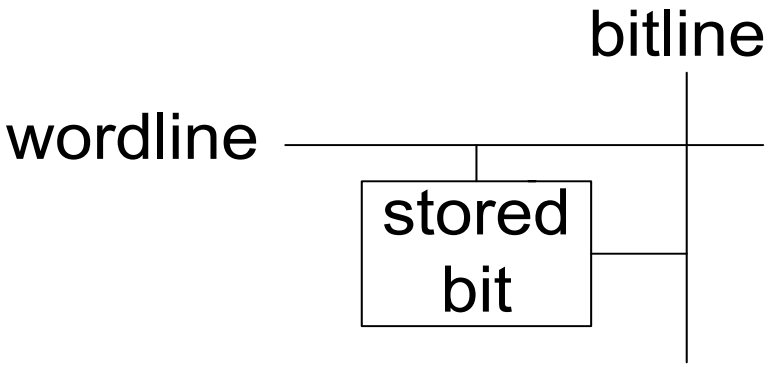


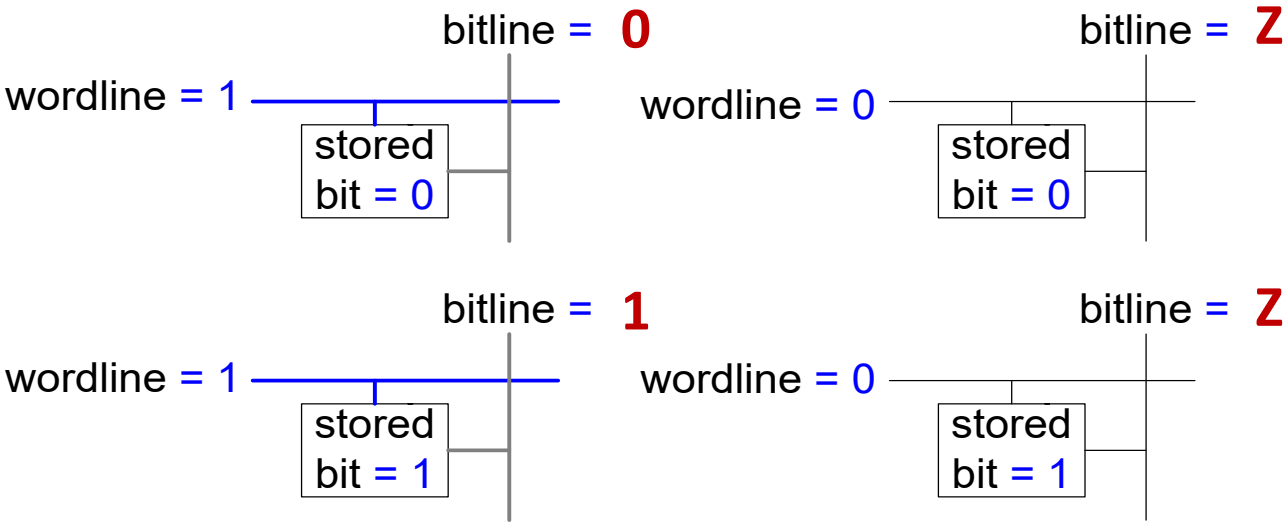
(a) (b)

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Memory Array Bit Cells





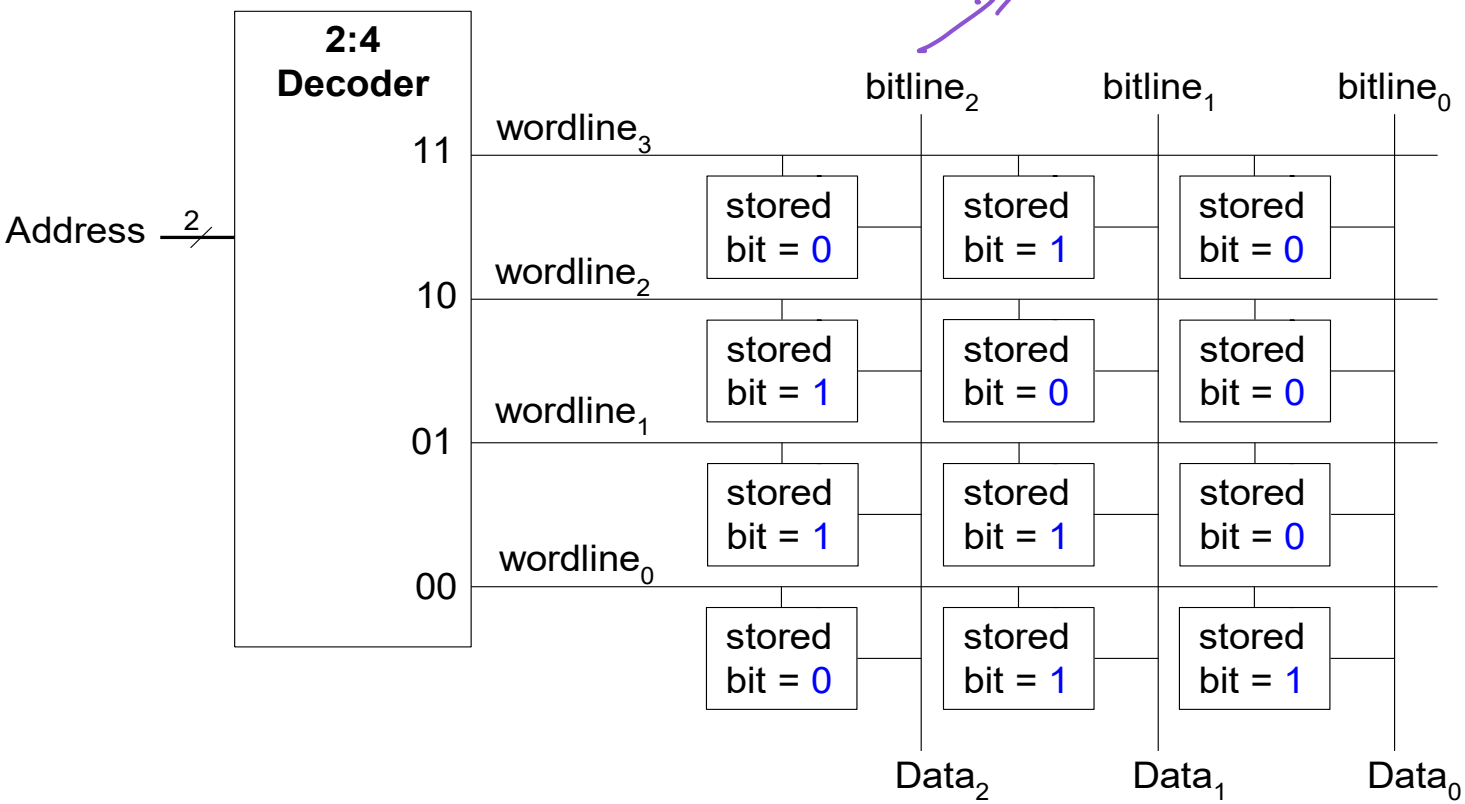
(a) (b)

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Memory Array

- **Wordline:**
  - like an enable
  - single row in memory array read/written
  - corresponds to unique address
  - only one wordline HIGH at once

that's very cool!



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Types of RAM

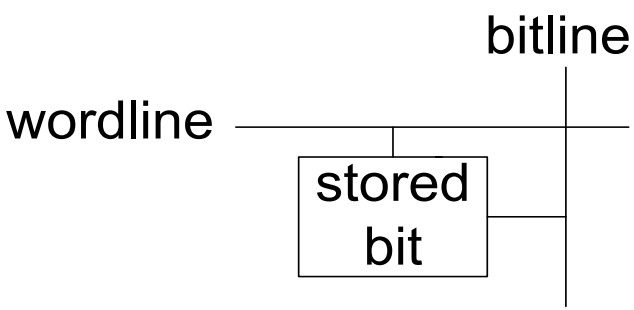
- **DRAM** (Dynamic random access memory)
- **SRAM** (Static random access memory)
- Differ in how they store data:
  - DRAM uses a capacitor
  - SRAM uses cross-coupled inverters

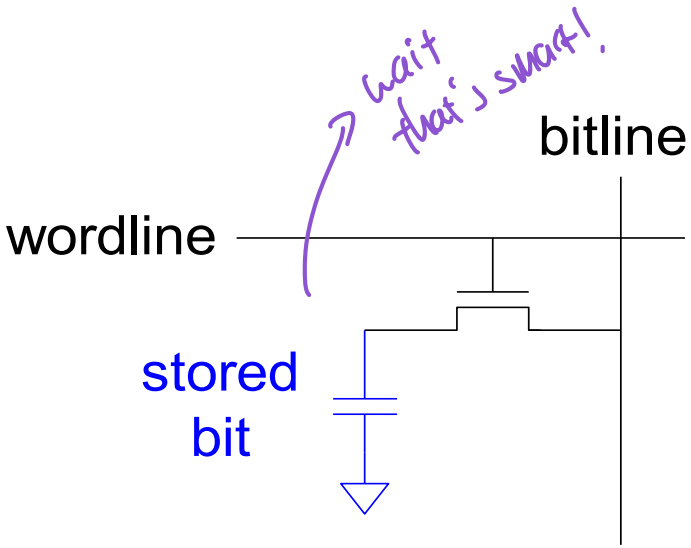
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

DRAM

- Data bits stored on capacitor
- *Dynamic* because the value needs to be refreshed (rewritten) periodically and after read:
  - Charge leakage from the capacitor degrades the value
  - Reading destroys the stored value

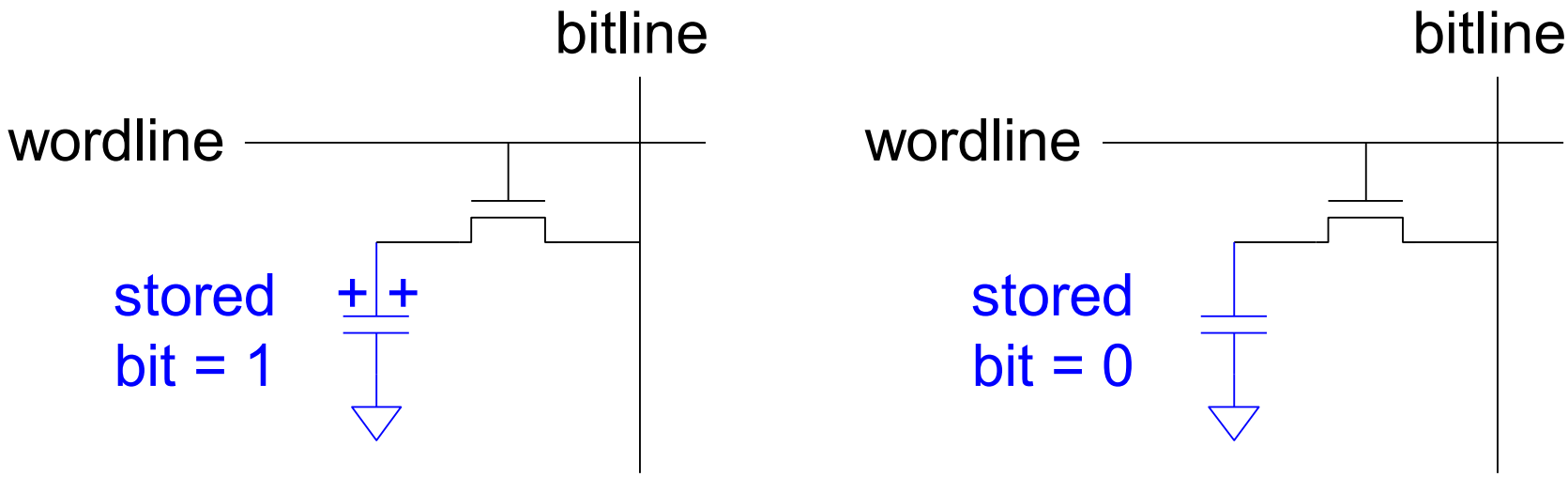


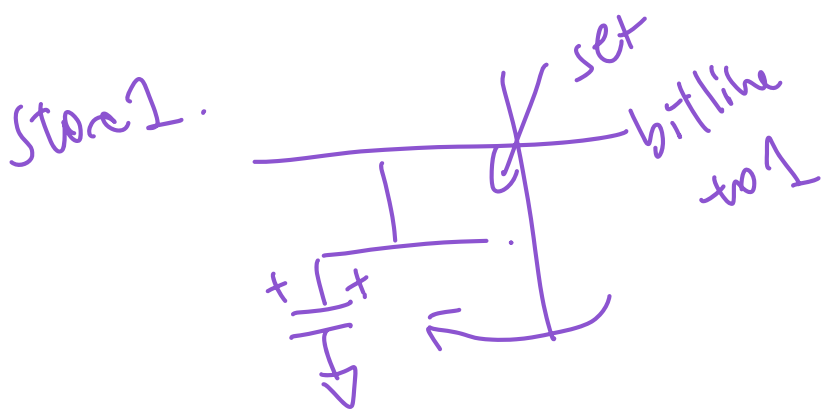


© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

DRAM

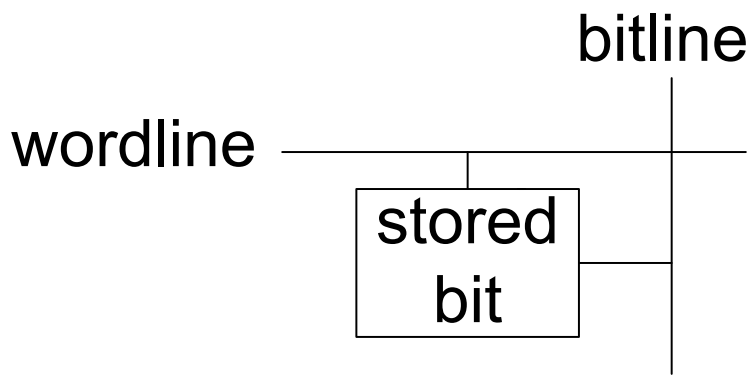




© *Digital Design and Computer Architecture*, 2nd Edition, 2012

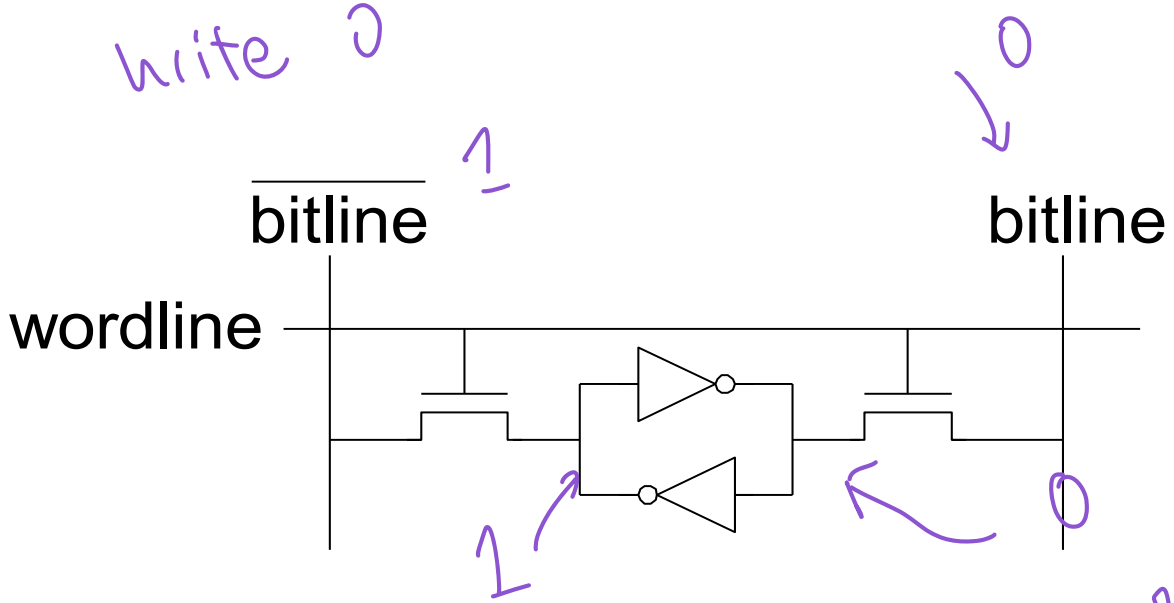
ELSEVIER

SRAM



Driver

write 0

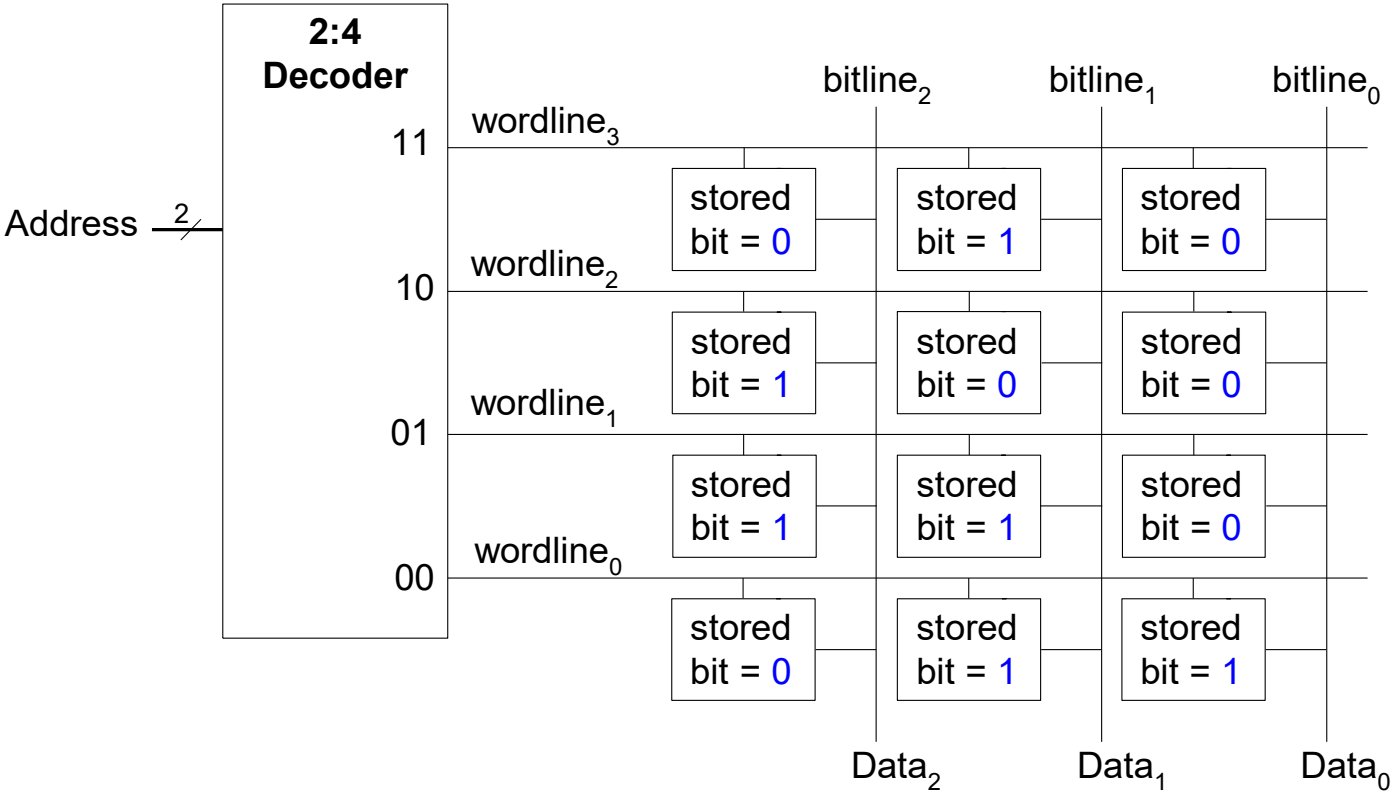


1 & 0 stand as voltage difference?

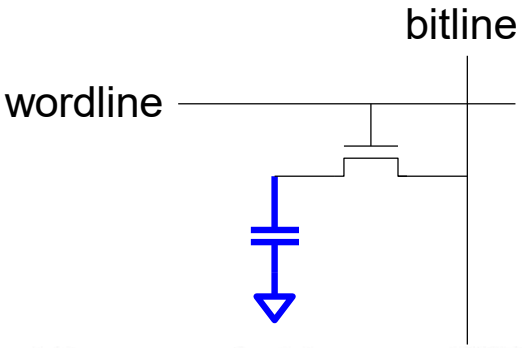
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

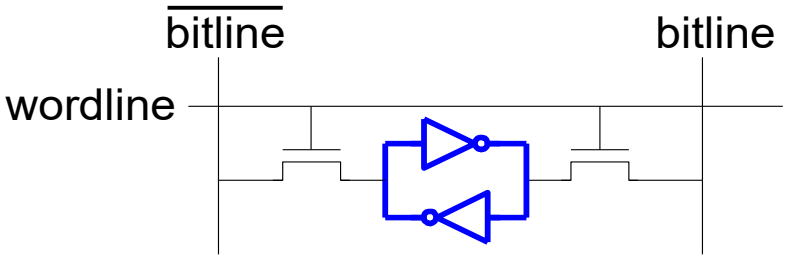
Memory Arrays



DRAM bit cell:



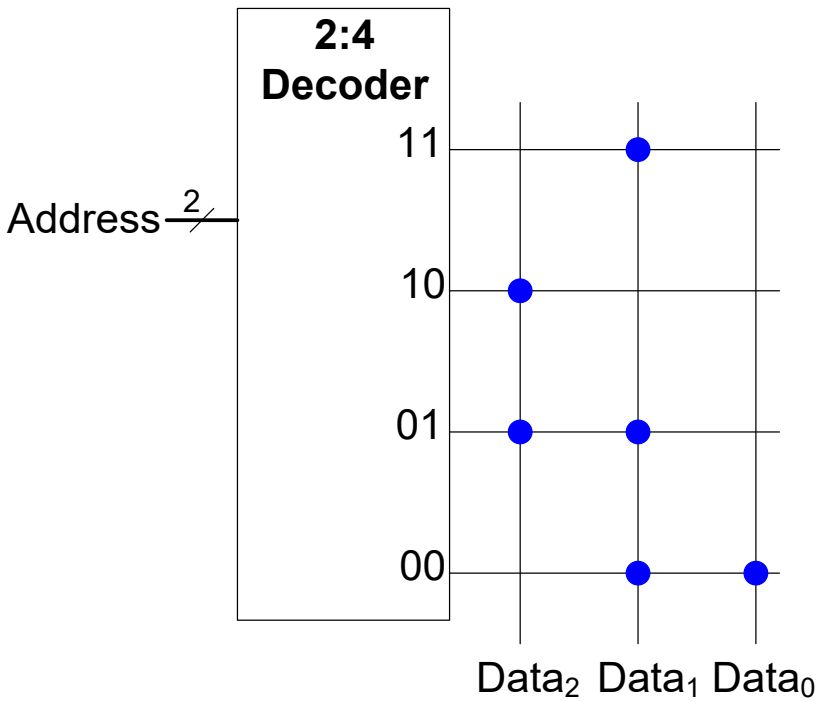
SRAM bit cell:

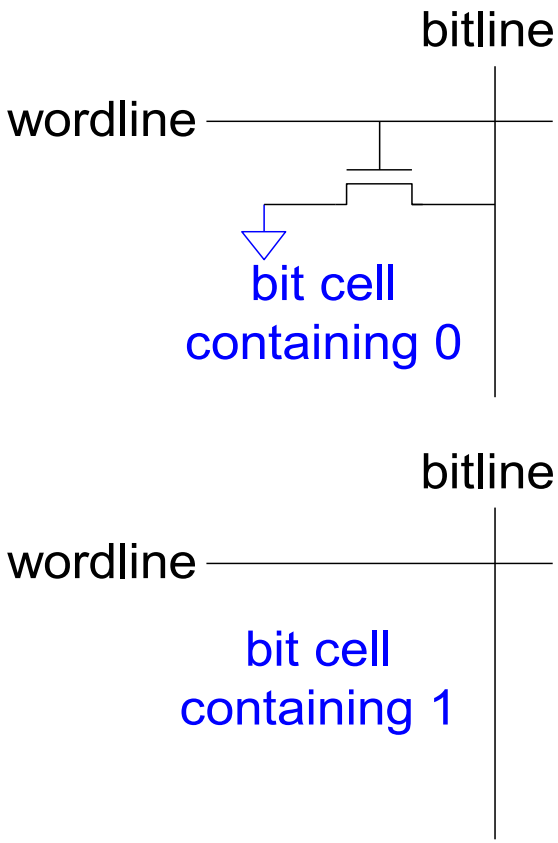


© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

ROM: Dot Notation

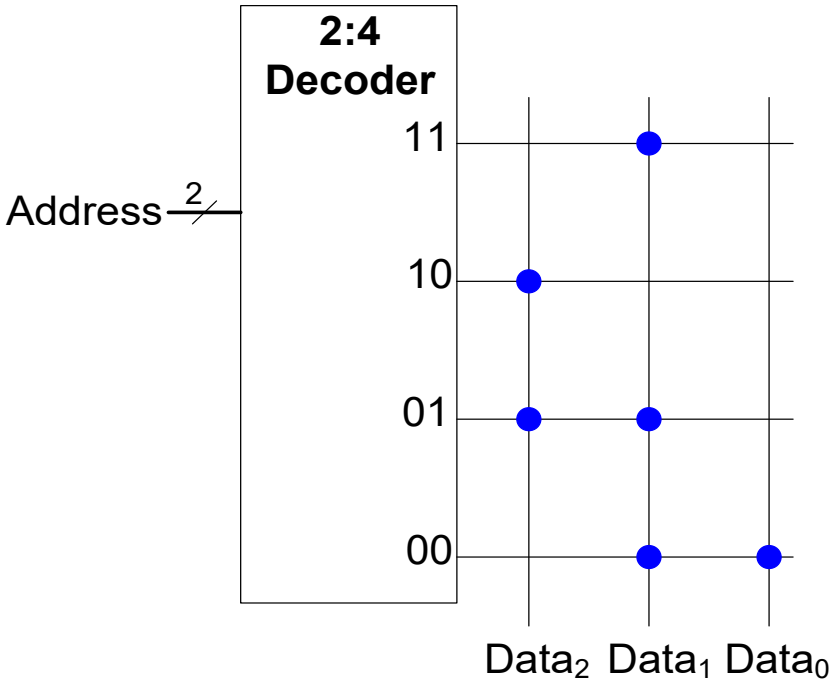




© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

ROM Storage



Address	Data		
11	0	1	0
10	1	0	0
01	1	1	0
00	0	1	1

depth

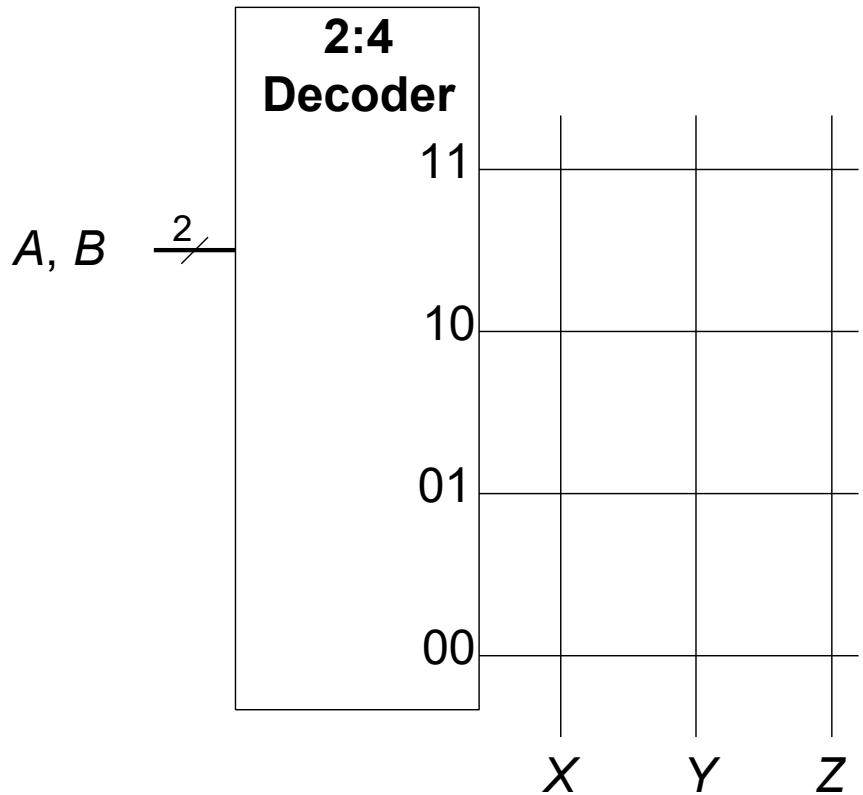
width

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Example: Logic with ROMs

Implement the following logic functions using a $2^2 \times 3$-bit ROM:

- $X = AB$
- $Y = A + B$
- $Z = A\overline{B}$

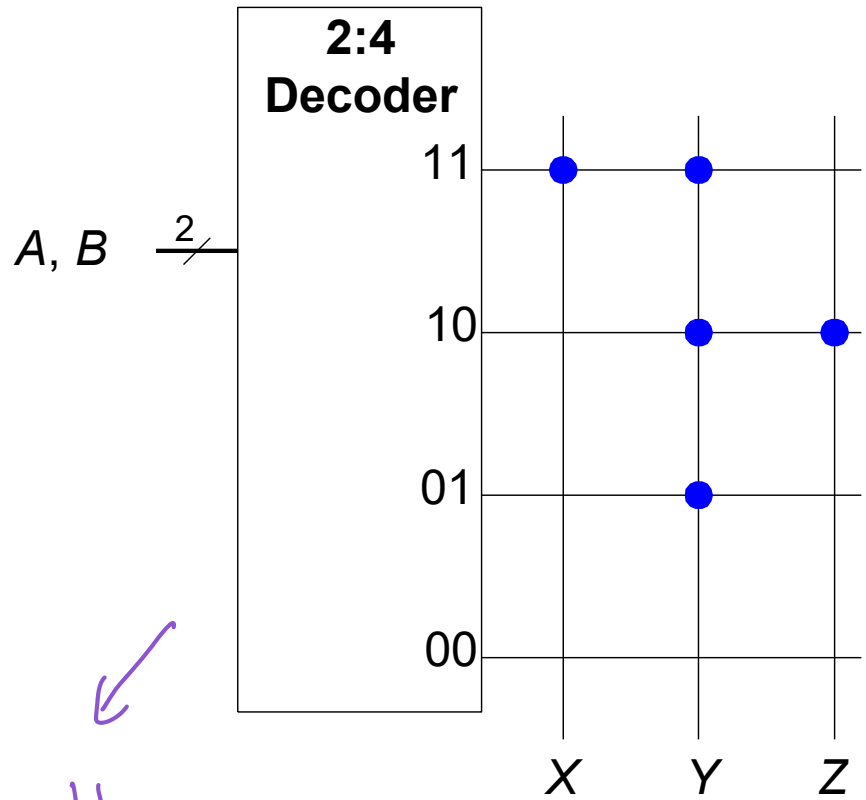


© Digital Design and Computer Architecture, 2nd Edition, 2012

Example: Logic with ROMs

Implement the following logic functions using a $2^2 \times 3$-bit ROM:

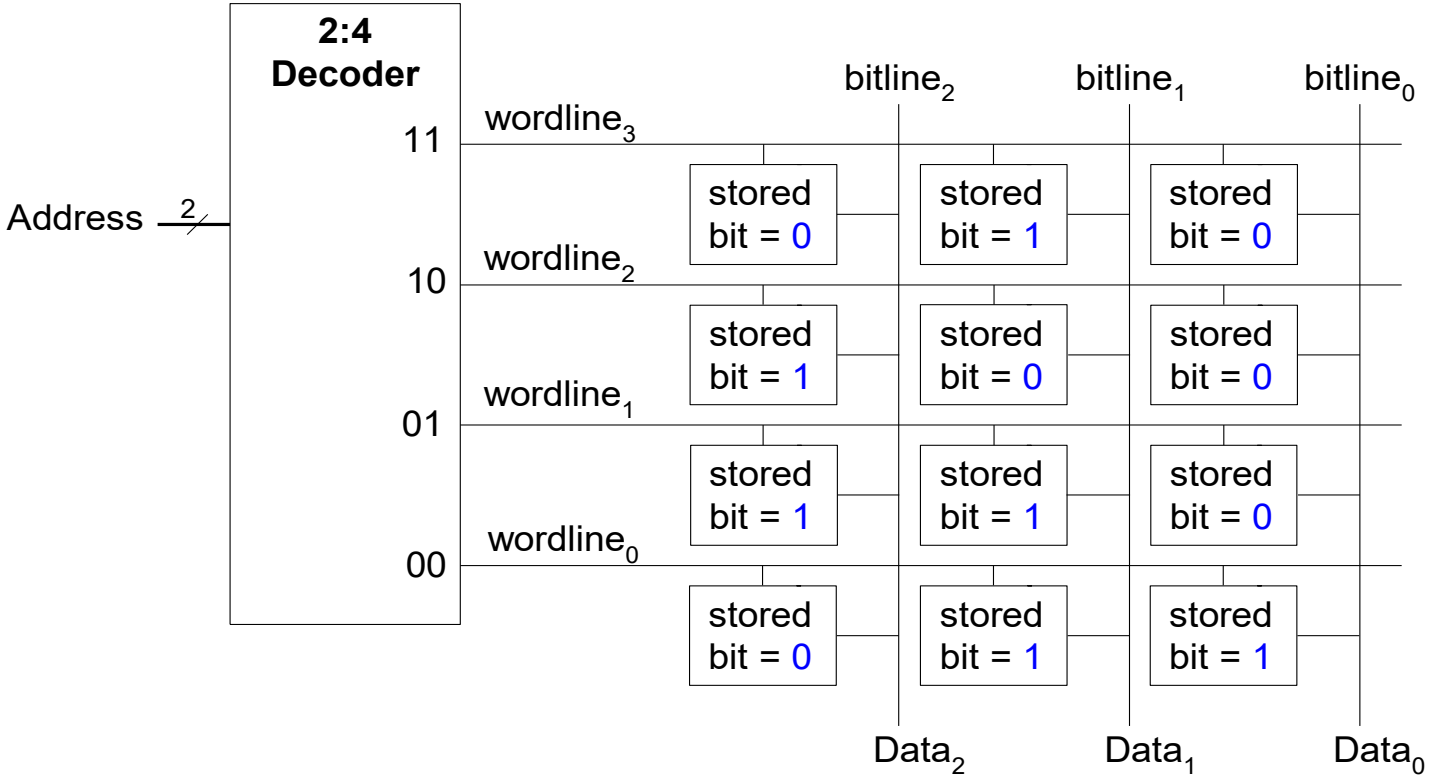
- $X = AB$
- $Y = A + B$
- $Z = A\overline{B}$



that's cool!

© Digital Design and Computer Architecture, 2nd Edition, 2012

Logic with Any Memory Array



$$Data_2 = A_1 \oplus A_0$$

$$Data_1 = \overline{A}_1 + A_0$$

$$Data_0 = \overline{A}_1 \overline{A}_0$$

© Digital Design and Computer Architecture, 2nd Edition, 2012

ELSEVIER

Logic with Memory Arrays

Implement the following logic functions using a $2^2 \times 3$-bit memory array:

- $X = AB$
- $Y = A + B$
- $Z = A\overline{B}$

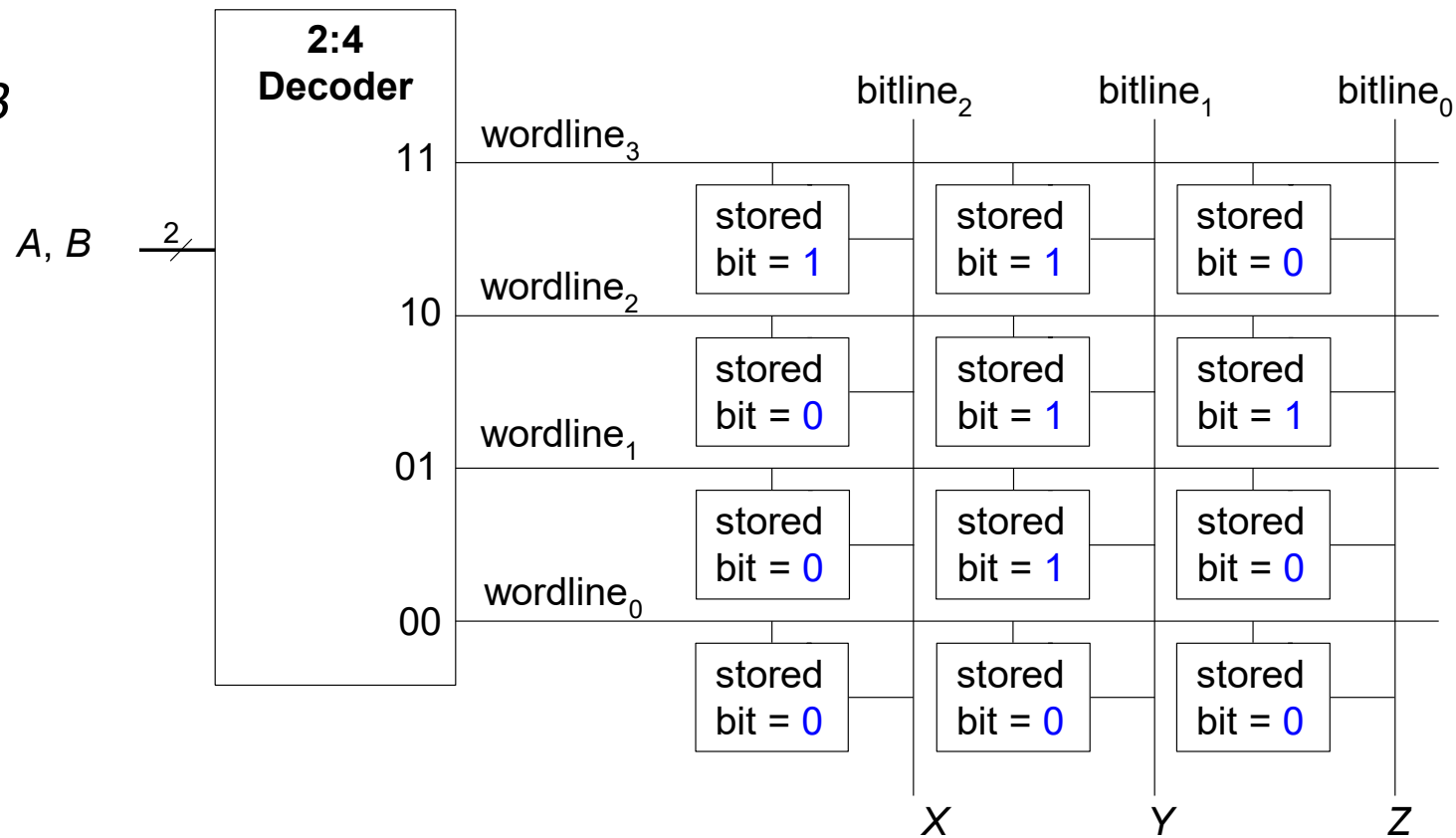
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Logic with Memory Arrays

Implement the following logic functions using a $2^2 \times 3$-bit memory array:

- $X = AB$
- $Y = A + B$
- $Z = A\overline{B}$



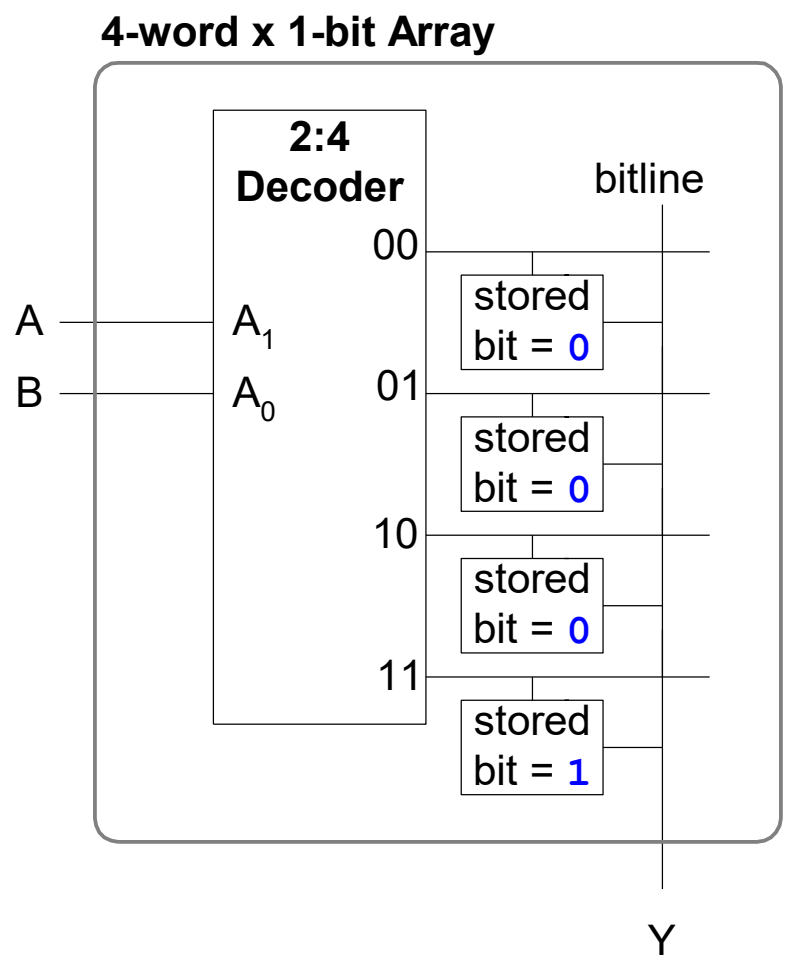
© Digital Design and Computer Architecture, 2nd Edition, 2012

Logic with Memory Arrays

Called *lookup tables* (LUTs): look up output at each input combination (address)

Truth Table

A	B	Y
0	0	**0**
0	1	**0**
1	0	**0**
1	1	**1**



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Logic Arrays

- **PLAs** (Programmable logic arrays)
  - AND array followed by OR array
  - Combinational logic only
  - Fixed internal connections

(combinational logic)

can use PLAs to implement sequential logic ?

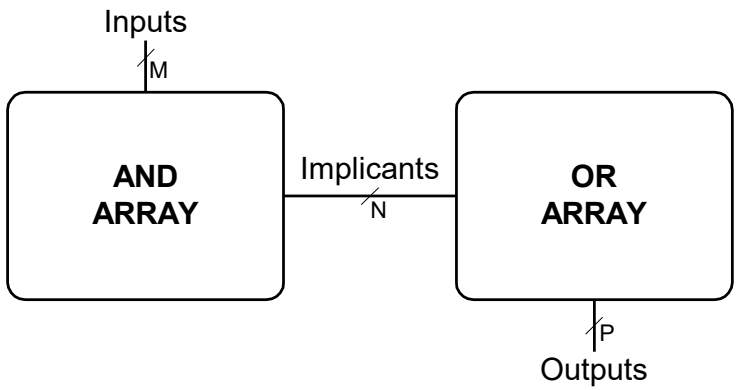
- **FPGAs** (Field programmable gate arrays)
  - Array of Logic Elements (LEs)
  - Combinational and sequential logic
  - Programmable internal connections

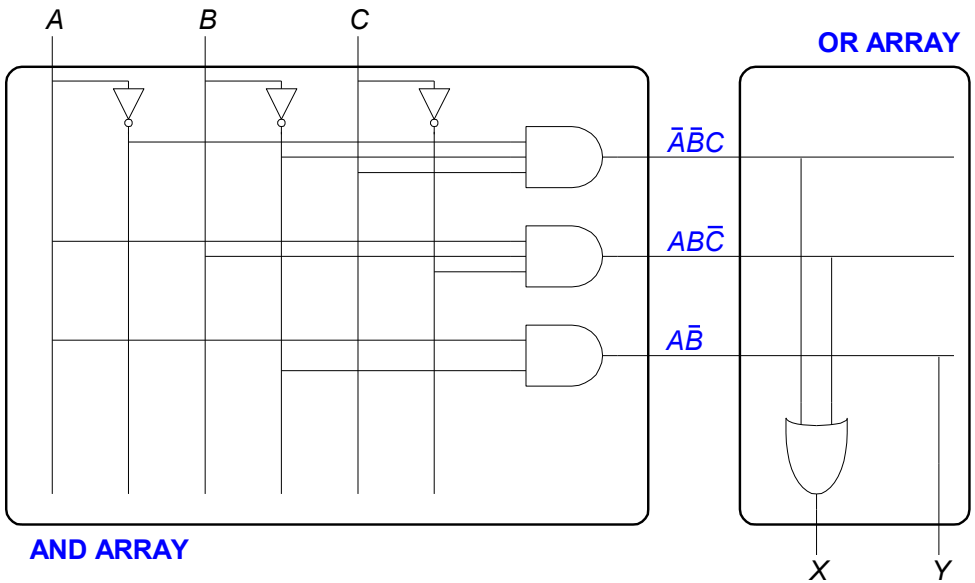
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

PLAs

- $X = \overline{A}\,\overline{B}C + AB\overline{C}$ → basically using sum of products.
- $Y = A\overline{B}$

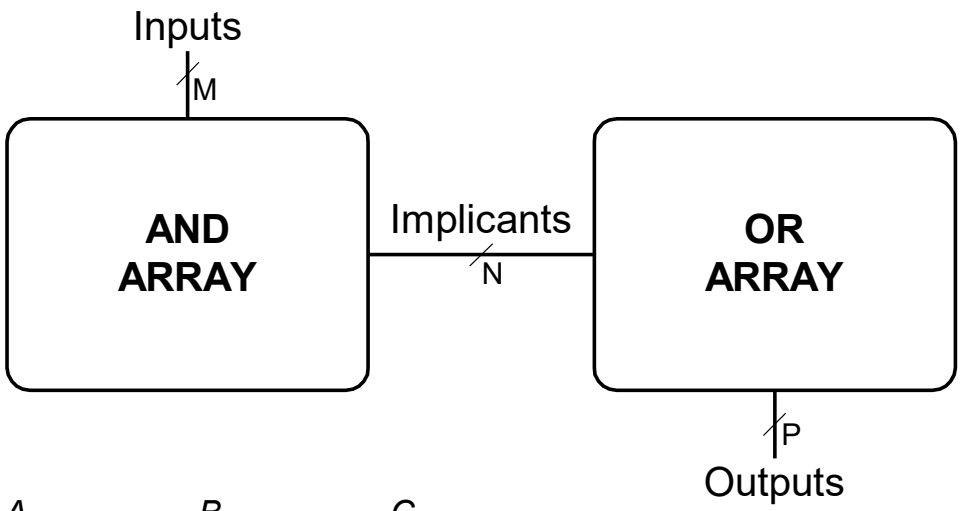


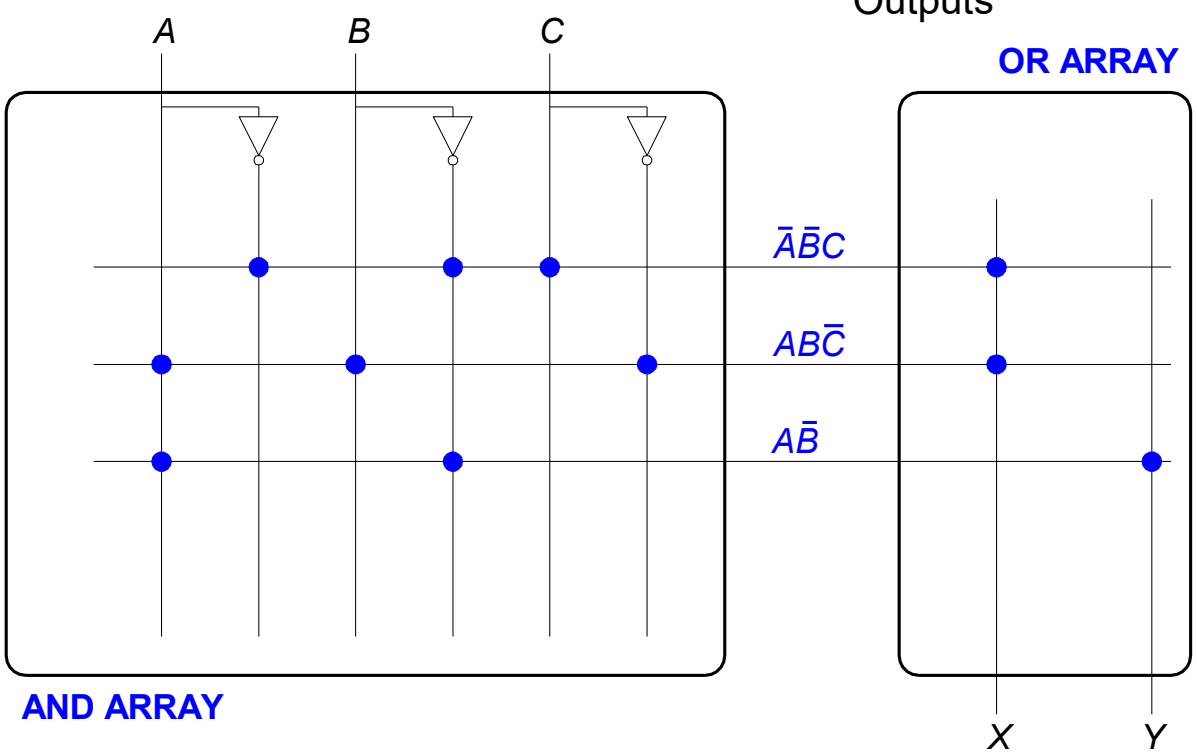


© Digital Design and Computer Architecture, 2nd Edition, 2012

ELSEVIER

PLAs: Dot Notation





© Digital Design and Computer Architecture, 2nd Edition, 2012

ELSEVIER